

Department of Computer Science and Engineering (AIML)
Database Management Systems (DBMS) Project Report

WORKFORCE MANAGEMENT SYSTEM

Attendance, Shift, and Leave Tracking

Submitted by	S Sri Shanmukh
Roll Number	25WU0102268
Course	AIML Whales
Faculty Name	A Kiranmayee

Contents

1. Cover Page
2. Abstract
3. Introduction and Problem Statement
4. Objectives and Scope
5. Software and Hardware Requirements
6. ER Diagram
7. Relational Schema and Normalization
8. Data Dictionary
9. SQL Commands Used (DDL, DML) with Sample Outputs
10. Queries with Outputs (including Presentation-II Query)
11. UI Design and Screenshots
12. Implementation Details
13. Testing
14. Conclusion and Future Enhancements
15. References
16. Appendix: GitHub Repository Link

2. Abstract

The Workforce Management System is a comprehensive database-driven application designed to automate and streamline core Human Resources operations. This project transitions traditional manual employee tracking into a centralized digital platform. The core database schema handles employee records, shift scheduling, real-time attendance logging, and leave request workflows. The system ensures data integrity and is accessed through a hybrid frontend/middleware stack built with Node.js and Python, offering efficient API routing and an intuitive interface for both administrators and employees.

At its heart, the system is a normalized relational database consisting of five inter-related tables: Department, Shift, Employee, Attendance, and Leave_Request. Primary keys, foreign keys, unique constraints, and NOT NULL rules are used throughout so that every record is identifiable, every relationship is enforceable, and invalid or orphaned data is rejected by the database engine itself rather than by application code alone. The schema is designed up to Third Normal Form (3NF) to eliminate redundancy and update anomalies.

On top of the database, a Node.js middleware layer exposes REST-style API endpoints that execute parameterized SQL queries, while a Python-based frontend presents dashboards, attendance registers, shift tables, and a leave management console. Administrators can monitor live metrics such as the number of employees present today, employees on leave, and pending leave approvals, and can run analytical queries such as department-wide overtime and approved sick leaves. The project demonstrates how core DBMS concepts-ER modelling, relational mapping, normalization, SQL (DDL, DML, DQL), and constraint enforcement-can be applied to solve a real organizational problem.

Keywords: Workforce Management, Attendance Tracking, Shift Scheduling, Leave Management, Relational Database, MySQL, Normalization, Node.js, Python.

3. Introduction and Problem Statement

3.1 Introduction

Managing workforce logistics in an organization requires strict data integrity and efficient business logic. This project provides a robust relational database backend to track employees across various departments and shifts while logging daily attendance and managing leave approvals.

Every organization, regardless of its size, must answer a few recurring questions each day: Who is supposed to be at work? Who actually turned up, and when? Who is on approved leave? Which employees worked beyond their scheduled hours? In small teams these questions are often answered with paper registers or spreadsheets, but as the headcount grows across multiple departments and shifts, such approaches quickly become unreliable. A database management system provides a principled way to store this information once, relate it correctly, and query it on demand.

The Workforce Management System developed in this project models the day-to-day HR workflow as a set of related entities. Employees belong to departments and are assigned to shifts. Each working day, an employee's clock-in and clock-out times are recorded as an attendance log. When an employee needs time off, a leave request is raised and moves through the Pending, Approved, or Rejected states. By relating these entities through primary and foreign keys, the system can answer complex questions-such as "which employees in the Sales department worked overtime?"-with a single SQL query.

3.2 Problem Statement

Manual HR tracking systems are prone to human error, data redundancy, and inefficiencies in payroll calculation. This project replaces manual tracking to streamline employee attendance, shifts, and leaves, strictly enforcing data integrity and operational business rules.

The specific shortcomings of manual or spreadsheet-based tracking that this project addresses are listed below.

- **Human error:** Hand-written registers and manually typed spreadsheets lead to wrong times, missing entries, and illegible records.
- **Data redundancy:** The same employee or department information is repeated in many places, so a single change (for example, a department rename) must be made everywhere or the data becomes inconsistent.

- **Payroll inefficiency:** Calculating overtime, absences, and paid or unpaid leave from unstructured records is slow and error-prone.
- **Weak approval tracking:** Leave requests sent through email or messaging apps are easily lost, and there is no single place to see which are pending, approved, or rejected.
- **Poor analytics:** Summaries such as department-wise overtime or approved sick leaves require manual counting and cannot be produced instantly.

4. Objectives and Scope

4.1 Objectives

- **Automate daily attendance:** Record employee Clock-In and Clock-Out automatically, with a status for each day (for example Present, Late, Overtime, Half-Day, Absent, or On Leave).
- **Manage shifts:** Define shifts with start and end times and assign specific shifts to employees seamlessly.
- **Process leave requests:** Capture leave requests with type, date range, and reason, and track each through the Approved, Pending, and Rejected states.
- **Enforce integrity:** Enforce relational constraints to maintain data integrity (e.g., circular dependencies between departments and managers).
- **Provide analytics:** Support queries for analytics such as department-wide overtime and approved sick leaves.
- **Offer a usable interface:** Present the data through a simple web interface so that administrators and employees do not need to write SQL.

4.2 Scope

The scope encompasses the design and implementation of a centralized relational database handling five core entities: Departments, Shifts, Employees, Attendance Logs, and Leave Requests. It supports querying for analytics like department-wide overtime and approved sick leaves.

Within scope

- Design of the ER diagram, relational schema, and 3NF normalization for the five core entities.
- Creation of the database using SQL DDL, including primary keys, foreign keys, unique and not-null constraints.
- Insertion, update, deletion, and retrieval of data using SQL DML and DQL, including joins and aggregate queries.
- A Node.js middleware (API) layer and a Python-based user interface for administrators and employees.
- Dashboards and tables for attendance, shifts, and leave management.

Outside scope (planned as future enhancements)

- Automatic payroll and salary computation.
- Biometric or RFID hardware integration for clock-in and clock-out.
- Multi-branch or multi-company (multi-tenant) deployments.

5. Software and Hardware Requirements

5.1 Software Requirements

Component	Technology / Tool	Purpose
Database	MySQL or PostgreSQL	Stores all relational data and enforces integrity constraints
Backend / Middleware	Node.js	API routing and execution of parameterized SQL queries
Frontend	Python (Streamlit / Flask / Tkinter)	User interface for administrators and employees
IDE / Tools	Visual Studio Code, MySQL Workbench / pgAdmin	Code editing, schema design, and query execution
Operating System	Windows 10/11, Linux, or macOS	Development and deployment platform
Version Control	Git and GitHub	Source code management and project hosting

The system was developed and tested with MySQL as the database server. Because only standard SQL features are used for the core schema, the same design can be ported to PostgreSQL with minor syntax changes (for example, replacing `AUTO_INCREMENT` with `SERIAL` or `IDENTITY` columns).

5.2 Hardware Requirements

Component	Minimum Requirement	Recommended
Processor	Intel Core i3 or equivalent	Intel Core i5 / Ryzen 5 or higher
RAM	4 GB	8 GB or more
Storage	2 GB free disk space	10 GB free (SSD preferred)
Display	1366 × 768 resolution	1920 × 1080 resolution
Network	Local network / localhost	Broadband connection for hosted deployment

For a future production deployment with attendance hardware, the following would also be required: an ESP32 microcontroller or RFID reader at each entry gate, and a server (on-premise or cloud) hosting the database and API.

6. ER Diagram

The Entity-Relationship (ER) diagram below models the five entities of the system and the relationships between them. Rectangles represent entities, ellipses represent attributes (underlined attributes are primary keys, and attributes marked FK are foreign keys), and diamonds represent relationships with their cardinalities shown as (1) and (N).

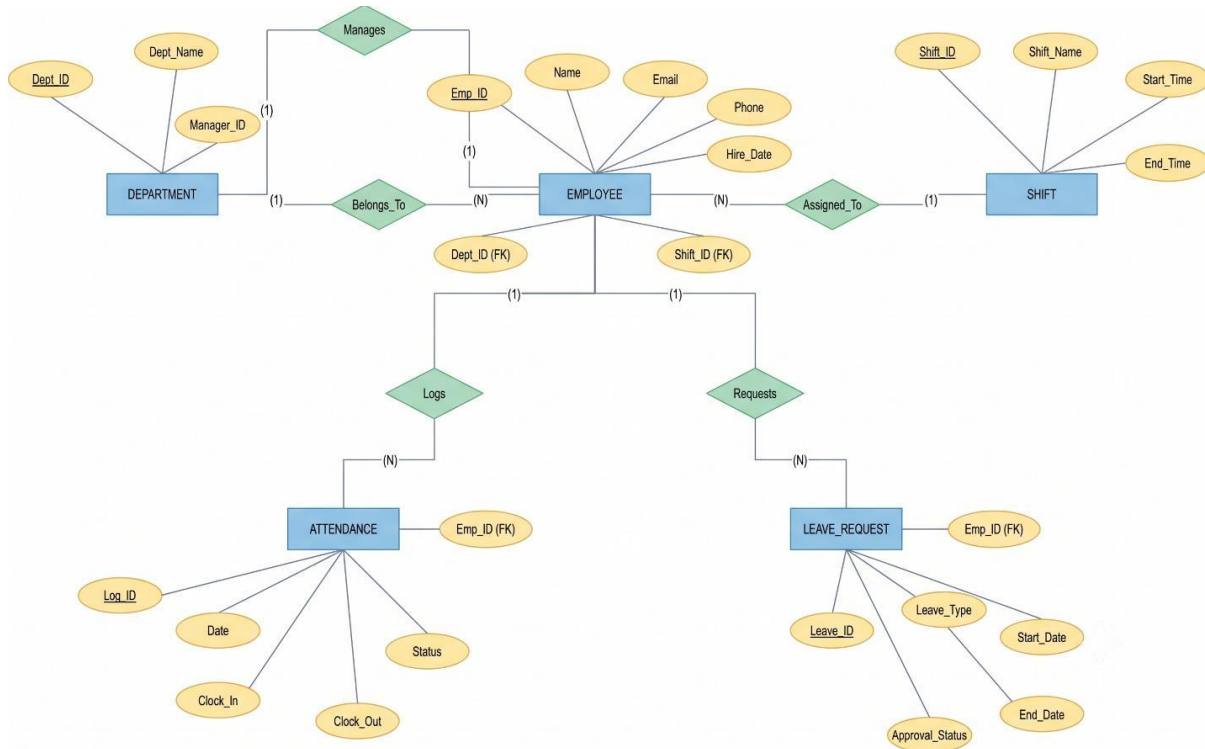


Figure 6.1: ER diagram of the Workforce Management System

6.1 Entities and Their Roles

- **DEPARTMENT:** Represents an organizational unit (such as Sales or IT & Security). Attributes are Dept_ID, Dept_Name, and Manager_ID.
- **SHIFT:** Represents a working time-slot. Attributes are Shift_ID, Shift_Name, Start_Time, and End_Time.
- **EMPLOYEE:** The central entity. Attributes are Emp_ID, Name, Email, Phone, Hire_Date, together with the foreign keys Dept_ID and Shift_ID.
- **ATTENDANCE:** A daily log for an employee. Attributes are Log_ID, Date, Clock_In, Clock_Out, Status, and the foreign key Emp_ID.
- **LEAVE_REQUEST:** A request for time off. Attributes are Leave_ID, Leave_Type, Start_Date, End_Date, Approval_Status, and the foreign key Emp_ID.

6.2 Relationships

- **Belongs_To (DEPARTMENT – EMPLOYEE):** each employee belongs to exactly one department, and a department has many employees.
- **Manages (DEPARTMENT – EMPLOYEE):** each department is managed by one employee (via Manager_ID), and the diagram shows this as a 1:1 relationship. Together with Belongs_To, this creates a circular dependency between the two tables, which is handled explicitly in the DDL (see Section 9).
- **Assigned_To (EMPLOYEE – SHIFT):** each employee is assigned one shift, and a shift can have many employees.
- **Logs (EMPLOYEE – ATTENDANCE):** one employee generates many attendance logs over time.
- **Requests (EMPLOYEE – LEAVE_REQUEST):** one employee can raise many leave requests.

6.3 Relational Mapping Overview

- DEPARTMENT to EMPLOYEE: 1-to-M (One department employs many employees)
- SHIFT to EMPLOYEE: 1-to-M (One shift has many assigned employees)
- EMPLOYEE to ATTENDANCE: 1-to-M (One employee has many attendance logs)
- EMPLOYEE to LEAVE_REQUEST: 1-to-M (One employee submits many leave requests)

In the mapping from ER model to relational model, each 1-to-M relationship is implemented by placing the primary key of the “one” side as a foreign key in the table of the “many” side. Hence Dept_ID and Shift_ID appear in EMPLOYEE, while Emp_ID appears in both ATTENDANCE and LEAVE_REQUEST.

7. Relational Schema and Normalization

7.1 Relational Schema

A relational schema acts as the structural blueprint for a database, defining exactly how data is organized, stored, and logically interconnected. It provides the architectural foundation for a Relational Database Management System (RDBMS) without containing the actual data itself.

A standard relational schema consists of the following core components:

- **Relations (Tables):** The fundamental structures that represent distinct entities (such as EMPLOYEE or DEPARTMENT), organized into rows and columns.
- **Attributes (Columns):** The specific properties or data points that define the entity, such as Emp_ID, Name, or Hire_Date.
- **Domains (Data Types):** The acceptable format and range of values for each attribute, ensuring data consistency (e.g., INT for whole numbers, VARCHAR for text).
- **Primary Keys:** A specific attribute (or combination of attributes) that uniquely identifies each individual record within a table, ensuring no duplicate rows exist.
- **Foreign Keys:** Attributes that create referential links between different tables. They establish relationships (like 1-to-Many) and map how data in one table corresponds to data in another.
- **Integrity Constraints:** Mandatory rules built into the schema (such as NOT NULL or UNIQUE) to enforce business logic and prevent the entry of invalid or orphaned data.

Relational schema of the Workforce Management System

In the notation below, underlined concepts are described in brackets: PK denotes a primary key and FK denotes a foreign key.

```
DEPARTMENT (Dept_ID PK, Dept_Name, Manager_ID FK -> EMPLOYEE.Emp_ID)
SHIFT      (Shift_ID PK, Shift_Name, Start_Time, End_Time)
EMPLOYEE   (Emp_ID PK, Name, Email, Phone, Hire_Date,
            Dept_ID FK -> DEPARTMENT.Dept_ID,
            Shift_ID FK -> SHIFT.Shift_ID)
ATTENDANCE (Log_ID PK, Emp_ID FK -> EMPLOYEE.Emp_ID,
            Date, Clock_In, Clock_Out, Status)
LEAVE_REQUEST(Leave_ID PK, Emp_ID FK -> EMPLOYEE.Emp_ID,
            Leave_Type, Start_Date, End_Date, Approval_Status)
```

7.2 Normalization

Normalization is the process of organizing the columns and tables of a relational database to reduce redundancy and avoid insertion, update, and deletion anomalies. The database is in Third Normal Form (3NF).

- **1NF:** All attributes are atomic (e.g., Name is a single string, Start_Time is a distinct time format). There are no repeating groups or multi-valued attributes, and every table has a primary key.
- **2NF:** No partial dependencies exist. All non-key attributes depend entirely on the primary key of their respective tables. Because every table uses a single-column surrogate primary key (an auto-increment integer), partial dependency on part of a composite key cannot occur.
- **3NF:** No transitive dependencies. E.g., Dept_Name depends on Dept_ID in the DEPARTMENT table, rather than being stored redundantly in the EMPLOYEE table.

Illustration of the 3NF decision

If EMPLOYEE stored Dept_Name directly, then $\text{Emp_ID} \rightarrow \text{Dept_ID}$ and $\text{Dept_ID} \rightarrow \text{Dept_Name}$ would give the transitive dependency $\text{Emp_ID} \rightarrow \text{Dept_Name}$. Renaming a department would then require updating every employee row of that department, and a department with no employees could not be stored at all. By keeping Dept_Name only in DEPARTMENT and referring to it through the foreign key Dept_ID, a rename is a single-row update. The same reasoning is applied to shift timings, which live only in the SHIFT table.

Every determinant in the schema is a candidate key (Emp_ID, Dept_ID, Shift_ID, Log_ID, and Leave_ID, plus the unique attributes Email, Phone, Dept_Name, and Shift_Name), so the relations also satisfy Boyce-Codd Normal Form (BCNF) for the functional dependencies identified.

8. Data Dictionary

The data dictionary below documents the structure of each table: its attributes, the data type (domain) of each attribute, and the constraints applied to it.

Table 8.1: Department

Attribute	Data Type	Constraints
Dept_ID	INT	Primary Key, Auto-Increment, Not NULL
Dept_Name	VARCHAR(50)	Unique, Not NULL
Manager_ID	INT	Not NULL (references EMPLOYEE.Emp_ID; added with ALTER TABLE)

Table 8.2: Shift

Attribute	Data Type	Constraints
Shift_ID	INT	Primary Key, Auto-Increment, Not NULL
Shift_Name	VARCHAR(50)	Unique, Not NULL
Start_Time	TIME	Not NULL
End_Time	TIME	Not NULL

Table 8.3: Employee

Attribute	Data Type	Constraints
Emp_ID	INT	Primary Key, Auto-Increment, Not NULL
Name	VARCHAR(100)	Not NULL
Email	VARCHAR(100)	Unique, Not NULL
Phone	VARCHAR(20)	Unique, Not NULL
Hire_Date	DATE	Not NULL
Dept_ID	INT	Foreign Key (DEPARTMENT.Dept_ID), Not NULL
Shift_ID	INT	Foreign Key (SHIFT.Shift_ID), Not NULL

Table 8.4: Attendance

Attribute	Data Type	Constraints
Log_ID	INT	Primary Key, Auto-Increment, Not NULL
Emp_ID	INT	Foreign Key (EMPLOYEE.Emp_ID), Not NULL
Date	DATE	Not NULL
Clock_In	TIME	Not NULL
Clock_Out	TIME	NULL Allowed (for active shifts)
Status	VARCHAR(20)	Not NULL

Table 8.5: Leave Request

Attribute	Data Type	Constraints
Leave_ID	INT	Primary Key, Auto-Increment, Not NULL
Emp_ID	INT	Foreign Key (EMPLOYEE.Emp_ID), Not NULL
Leave_Type	VARCHAR(50)	Not NULL
Start_Date	DATE	Not NULL
End_Date	DATE	Not NULL
Approval_Status	VARCHAR(20)	Not NULL

8.6 Business Rules Enforced

- An employee must belong to an existing department and have an existing shift (foreign keys, NOT NULL).
- Email, phone number, department name, and shift name must be unique.
- Clock_Out may be NULL only while the employee's shift is still active.
- Approval_Status is restricted to Pending, Approved, or Rejected, and a leave's End_Date cannot be earlier than its Start_Date (CHECK constraints).
- A department's manager must be an existing employee (foreign key on Manager_ID).

9. SQL Commands Used (DDL, DML) with Sample Outputs

9.1 Data Definition Language (DDL)

DDL statements define the structure of the database. A notable design challenge is the circular dependency between DEPARTMENT and EMPLOYEE: EMPLOYEE.Dept_ID references DEPARTMENT, while DEPARTMENT.Manager_ID references EMPLOYEE. Two tables cannot each reference the other at creation time, so DEPARTMENT is created first without the manager foreign key, EMPLOYEE is created next, and the Manager_ID foreign key is added afterwards with ALTER TABLE.

```
CREATE DATABASE workforce_db;
USE workforce_db;

-- 1. Department (manager foreign key is added later)
CREATE TABLE Department (
  Dept_ID INT AUTO_INCREMENT PRIMARY KEY,
  Dept_Name VARCHAR(50) NOT NULL UNIQUE,
  Manager_ID INT NOT NULL
);

-- 2. Shift
CREATE TABLE Shift (
  Shift_ID INT AUTO_INCREMENT PRIMARY KEY,
  Shift_Name VARCHAR(50) NOT NULL UNIQUE,
  Start_Time TIME NOT NULL,
  End_Time TIME NOT NULL
);

-- 3. Employee
CREATE TABLE Employee (
  Emp_ID INT AUTO_INCREMENT PRIMARY KEY,
  Name VARCHAR(100) NOT NULL,
  Email VARCHAR(100) NOT NULL UNIQUE,
  Phone VARCHAR(20) NOT NULL UNIQUE,
  Hire_Date DATE NOT NULL,
  Dept_ID INT NOT NULL,
  Shift_ID INT NOT NULL,
  CONSTRAINT fk_emp_dept FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID),
  CONSTRAINT fk_emp_shift FOREIGN KEY (Shift_ID) REFERENCES Shift(Shift_ID)
);

-- 4. Resolve the circular dependency
ALTER TABLE Department
  ADD CONSTRAINT fk_dept_manager
  FOREIGN KEY (Manager_ID) REFERENCES Employee(Emp_ID);

-- 5. Attendance
CREATE TABLE Attendance (
  Log_ID INT AUTO_INCREMENT PRIMARY KEY,
  Emp_ID INT NOT NULL,
  `Date` DATE NOT NULL,
  Clock_In TIME NOT NULL,
  Clock_Out TIME NULL,
  Status VARCHAR(20) NOT NULL,
  CONSTRAINT fk_att_emp FOREIGN KEY (Emp_ID) REFERENCES Employee(Emp_ID),
  CONSTRAINT chk_att_status CHECK (Status IN
    ('Present','Late','Overtime','Half-Day','Absent','On Leave'))
);
```

```
-- 6. Leave_Request
CREATE TABLE Leave_Request (
  Leave_ID      INT AUTO_INCREMENT PRIMARY KEY,
  Emp_ID        INT NOT NULL,
  Leave_Type    VARCHAR(50) NOT NULL,
  Start_Date    DATE NOT NULL,
  End_Date      DATE NOT NULL,
  Approval_Status VARCHAR(20) NOT NULL DEFAULT 'Pending',
  CONSTRAINT fk_leave_emp FOREIGN KEY (Emp_ID) REFERENCES Employee(Emp_ID),
  CONSTRAINT chk_leave_dates CHECK (End_Date >= Start_Date),
  CONSTRAINT chk_leave_status CHECK (Approval_Status IN
    ('Pending','Approved','Rejected'))
);
```

Other DDL commands used during development include DESCRIBE (to verify table structure), ALTER TABLE (to modify columns or constraints), CREATE INDEX (to speed up lookups on Attendance.Emp_ID and Attendance.Date), and DROP TABLE (to rebuild the schema during testing).

```
DESCRIBE Employee;
CREATE INDEX idx_att_emp_date ON Attendance (Emp_ID, 'Date');
CREATE INDEX idx_leave_status ON Leave_Request (Approval_Status);
```

9.2 Data Manipulation Language (DML)

DML statements insert, modify, and remove records. Because of the circular dependency, departments are inserted with a placeholder manager, employees are then inserted, and finally the real manager is set with an UPDATE. Alternatively, foreign key checks can be switched off temporarily while seeding, as shown below.

INSERT

```
SET FOREIGN_KEY_CHECKS = 0;

INSERT INTO Department (Dept_Name, Manager_ID) VALUES
('Sales', 1), ('IT & Security', 3), ('Human Resources', 4);

INSERT INTO Shift (Shift_Name, Start_Time, End_Time) VALUES
('General Day Shift', '09:00:00', '17:30:00'),
('Early Morning Shift', '06:00:00', '14:30:00'),
('Afternoon Swing Shift', '14:00:00', '22:30:00'),
('Night Operations Shift', '22:00:00', '06:30:00'),
('Weekend Support Shift', '08:00:00', '16:00:00'),
('Flexi Hours Shift', '10:00:00', '18:30:00');

INSERT INTO Employee (Name, Email, Phone, Hire_Date, Dept_ID, Shift_ID) VALUES
('Michael Scott', 'michael.scott@corp.com', '9000000001', '2018-03-12', 1, 1),
('Rachel Green', 'rachel.green@corp.com', '9000000002', '2020-06-01', 1, 1),
('Fiona Gallagher', 'fiona.gallagher@corp.com', '9000000003', '2019-09-15', 2, 4),
('Ross Geller', 'ross.geller@corp.com', '9000000004', '2017-01-20', 3, 1),
('Monica Geller', 'monica.geller@corp.com', '9000000005', '2021-11-08', 2, 3);

SET FOREIGN_KEY_CHECKS = 1;

INSERT INTO Attendance (Emp_ID, 'Date', Clock_In, Clock_Out, Status) VALUES
(1, '2026-10-05', '09:00:00', '19:30:00', 'Overtime');
```

```
(2, '2026-10-05', '09:05:00', '19:00:00', 'Overtime'),
(3, '2026-10-05', '22:00:00', '08:00:00', 'Overtime'),
(4, '2026-10-05', '09:00:00', '17:30:00', 'Present'),
(5, '2026-10-05', '14:20:00', '22:30:00', 'Late'),
(1, '2026-10-06', '09:00:00', '17:30:00', 'Present'),
(4, '2026-10-07', '09:00:00', NULL, 'Present');
```

```
INSERT INTO Leave_Request (Emp_ID, Leave_Type, Start_Date, End_Date, Approval_Status) VALUES
(2, 'Sick Leave', '2026-09-22', '2026-09-22', 'Approved'),
(5, 'Sick Leave', '2026-10-12', '2026-10-13', 'Pending'),
(1, 'Casual Leave', '2026-10-20', '2026-10-21', 'Approved'),
(3, 'Sick Leave', '2026-09-14', '2026-09-16', 'Approved'),
(4, 'Annual Leave', '2026-12-06', '2026-12-21', 'Pending');
```

SELECT (verifying inserted data)

```
SELECT Emp_ID, Name, Dept_ID, Shift_ID FROM Employee;
```

Emp_ID	Name	Dept_ID	Shift_ID
1	Michael Scott	1	1
2	Rachel Green	1	1
3	Fiona Gallagher	2	4
4	Ross Geller	3	1
5	Monica Geller	2	3

Output 9.1: Employees inserted into the database (5 rows)

UPDATE

When an employee finishes the day, the open attendance record is closed by setting Clock_Out.

A pending leave request is approved or rejected by updating Approval_Status.

```
-- Clock-out for an employee who is still clocked in
UPDATE Attendance
SET Clock_Out = '17:35:00'
WHERE Emp_ID = 4 AND 'Date' = '2026-10-07' AND Clock_Out IS NULL;

-- Approve a pending leave request
UPDATE Leave_Request
SET Approval_Status = 'Approved'
WHERE Leave_ID = 2;

-- Re-assign an employee to a different shift
UPDATE Employee SET Shift_ID = 6 WHERE Emp_ID = 5;
```

Sample output: Query OK, 1 row affected (0.01 sec). Rows matched: 1 Changed: 1 Warnings: 0.

DELETE

```
-- Remove a rejected or mistaken leave request
DELETE FROM Leave_Request WHERE Leave_ID = 5 AND Approval_Status = 'Pending';
```

Sample output: Query OK, 1 row affected (0.01 sec).

Attempting to delete a department or shift that is still referenced by employees is rejected by the database with a foreign key error (for example, “Cannot delete or update a parent row: a foreign key constraint fails”), which protects the system from orphaned records.

10. Queries with Outputs, including the Query given during Presentation-II

This section lists the analytical queries supported by the system. Query 1 is the query demonstrated during Presentation-II; its original screenshot from the MySQL command-line client is reproduced below. The outputs of the remaining queries are shown against the sample data inserted in Section 9.

Query 1 (Presentation-II): Employees who worked overtime, with their department

This query joins three tables-EMPLOYEE, DEPARTMENT, and ATTENDANCE-to list the name and department of every employee who has an attendance record with the status “Overtime”. It supports the department-wide overtime analytics that are an objective of the project.

```
SELECT
  e.Name,
  d.Dept_Name
FROM
  EMPLOYEE e
JOIN
  DEPARTMENT d ON e.Dept_ID = d.Dept_ID
JOIN
  ATTENDANCE a ON e.Emp_ID = a.Emp_ID
WHERE
  a.Status = 'Overtime';
```

```
mysql> SELECT
->     e.Name,
->     d.Dept_Name
-> FROM
->     EMPLOYEE e
-> JOIN
->     DEPARTMENT d ON e.Dept_ID = d.Dept_ID
-> JOIN
->     ATTENDANCE a ON e.Emp_ID = a.Emp_ID
-> WHERE
->     a.Status = 'Overtime';
```

Name	Dept_Name
Fiona Gallagher	IT & Security
Michael Scott	Sales
Rachel Green	Sales

```
3 rows in set (0.03 sec)
```

Figure 10.1: Output of the Presentation-II overtime query (3 rows in set)

The output shows three employees-Fiona Gallagher (IT & Security), Michael Scott (Sales), and Rachel Green (Sales)-who worked overtime. Each row is produced by matching an

employee to a department through Dept_ID and to an attendance log through Emp_ID, and then filtering the logs on Status.

Query 2: Approved sick leaves with employee and department

```
SELECT e.Name, d.Dept_Name, l.Start_Date, l.End_Date,
       DATEDIFF(l.End_Date, l.Start_Date) + 1 AS Days
FROM   Leave_Request l
JOIN   Employee e ON l.Emp_ID = e.Emp_ID
JOIN   Department d ON e.Dept_ID = d.Dept_ID
WHERE  l.Leave_Type = 'Sick Leave'
AND    l.Approval_Status = 'Approved';
```

Name	Dept_Name	Start_Date	End_Date	Days
Rachel Green	Sales	2026-09-22	2026-09-22	1
Fiona Gallagher	IT & Security	2026-09-14	2026-09-16	3

Output 10.2: Approved sick leaves (2 rows)

Query 3: Number of employees assigned to each shift (LEFT JOIN)

A LEFT JOIN is used so that shifts with no assigned employees are still listed with a count of zero. This is the same information displayed in the “Active Assignments” column of the Shift screen.

```
SELECT s.Shift_ID, s.Shift_Name, COUNT(e.Emp_ID) AS Assigned_Employees
FROM   Shift s
LEFT JOIN Employee e ON e.Shift_ID = s.Shift_ID
GROUP BY s.Shift_ID, s.Shift_Name
ORDER BY s.Shift_ID;
```

Shift_ID	Shift_Name	Assigned_Employees
1	General Day Shift	3
2	Early Morning Shift	0
3	Afternoon Swing Shift	1
4	Night Operations Shift	1
5	Weekend Support Shift	0
6	Flexi Hours Shift	0

Output 10.3: Employees per shift (6 rows)

Query 4: Pending leave requests awaiting approval

```
SELECT l.Leave_ID, e.Name, l.Leave_Type, l.Start_Date, l.End_Date
FROM   Leave_Request l
JOIN   Employee e ON e.Emp_ID = l.Emp_ID
WHERE  l.Approval_Status = 'Pending'
ORDER BY l.Start_Date;
```

Leave_ID	Name	Leave_Type	Start_Date	End_Date
2	Monica Geller	Sick Leave	2026-10-12	2026-10-13

Leave_ID	Name	Leave_Type	Start_Date	End_Date
5	Ross Geller	Annual Leave	2026-12-06	2026-12-21

Output 10.4: Pending leave requests (2 rows)

Query 5: Department-wise headcount

```
SELECT d.Dept_Name, COUNT(e.Emp_ID) AS Headcount
FROM   Department d
LEFT JOIN Employee e ON e.Dept_ID = d.Dept_ID
GROUP BY d.Dept_Name
ORDER BY Headcount DESC, d.Dept_Name;
```

Dept_Name	Headcount
IT & Security	2
Sales	2
Human Resources	1

Output 10.5: Employees per department (3 rows)

Query 6: Overtime records per department (aggregate)

```
SELECT d.Dept_Name, COUNT(*) AS Overtime_Logs
FROM   Attendance a
JOIN   Employee e ON a.Emp_ID = e.Emp_ID
JOIN   Department d ON e.Dept_ID = d.Dept_ID
WHERE  a.Status = 'Overtime'
GROUP BY d.Dept_Name
ORDER BY Overtime_Logs DESC;
```

Dept_Name	Overtime_Logs
Sales	2
IT & Security	1

Output 10.6: Overtime logs per department (2 rows)

Query 7: Employees currently clocked in (no Clock_Out yet)

```
SELECT e.Name, a.Clock_In
FROM   Attendance a
JOIN   Employee e ON e.Emp_ID = a.Emp_ID
WHERE  a.'Date' = '2026-10-07' AND a.Clock_Out IS NULL;
```

Name	Clock_In
Ross Geller	09:00:00

Output 10.7: Employees with an active shift (1 row)

Note: the outputs for Queries 2 to 7 are computed from the compact sample dataset in Section 9.2, whereas the screenshots in Section 11 come from the larger demonstration dataset loaded into the running application.

11. UI Design and Screenshots

The user interface (branded “ChronosDBMS” in the application) is a clean, web-based dashboard with a persistent left navigation panel grouping the modules into Core Operations (Dashboard, Mark Attendance, Leave Management), Database Entity Tables (Departments, Employees, Work Shifts, Shift Assignments, Leave Types), and SQL Reports. A top navigation bar offers quick access to the main screens and a “Log Attendance” action button. A dark SQL console strip at the bottom of every screen displays the last executed query, its parameters, the number of rows returned, and the execution time, which makes the link between the interface and the underlying SQL visible.

11.1 Dashboard

The dashboard gives administrators a real-time overview of the workforce. Four live metric cards are each computed by an aggregate SQL query: Total Employees (COUNT of the Employee table), Present Today (attendance records for the current date), On Leave Today (leave requests whose date range contains today), and Pending Leave Requests (leave requests with status Pending). Below the cards, the screen shows today’s attendance register—a join of Attendance, Employee, Department, and Shift—together with a panel of pending leave approvals.

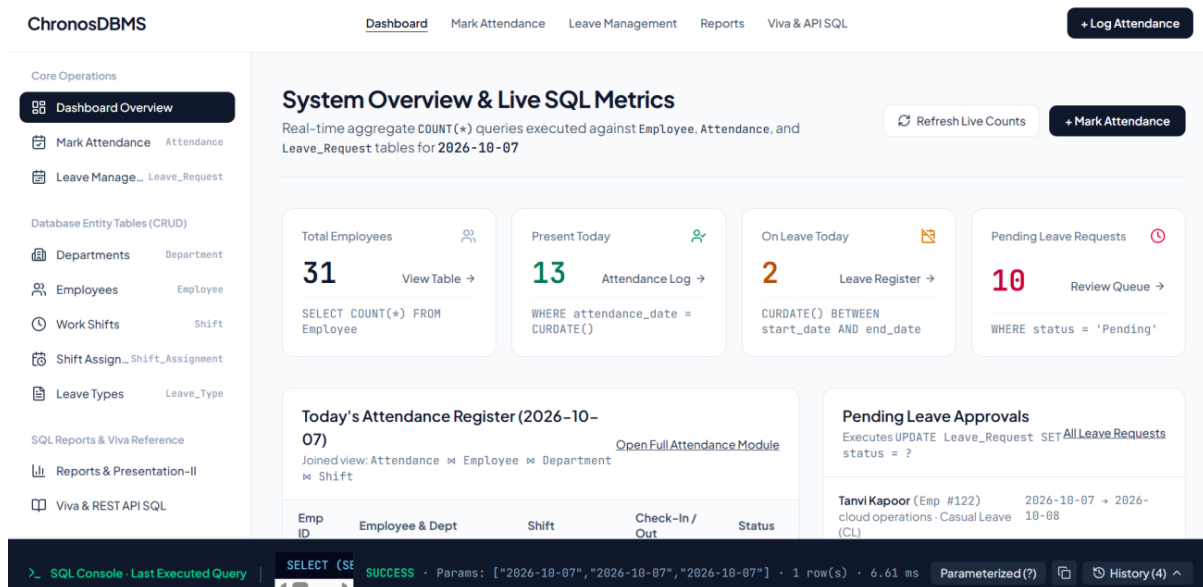


Figure 11.1: Dashboard showing live SQL metrics (31 employees, 13 present, 2 on leave, 10 pending requests)

11.2 Leave Management

The Leave Management screen lists all leave requests in a joined view of Leave_Request, Employee, Department, and Leave_Type (29 rows in the demonstration data). Each row shows

the request ID, the employee and department, the leave type and category, the date range, the number of days, the reason, and the status (Approved, Pending, or Rejected) in a colour-coded label. A search box and filter tabs (ALL, Pending, Approved, Rejected) allow administrators to narrow the list quickly.

Dashboard Mark Attendance **Leave Management** Reports Viva & API SQL [+ Log Attendance](#)

Leave Requests Joined View (29 rows)
SQL: SELECT lr.*, e.first_name, e.last_name, d.dept_name, lt.leave_name FROM Leave_Request lr INNER JOIN Employee e ... INNER JOIN Leave_Type lt

ALL Pending Approved Rejected

ID ↑	Employee (FK emp_id)	Leave Type (FK leave_type_id)	Date Range ↑	Days ↑	Reason	Status ↑
#32	Tarun O'Brien-Reddy (#111) Customer Success	Earned / Privilege Leave (EL) Paid Leave	2026-12-06 → 2026-12-21	16d	Annual leave	Approved
#31	Yash Agarwal (#123) Database & Infrastructure	Casual Leave (CL) Paid Leave	2026-10-13 → 2026-10-13	1d	House shifting	Pending
#29	Kavya Reddy (#118) Product Quality Assurance	Medical / Sick Leave (SL) Paid Leave	2026-10-05 → 2026-10-05	1d	Stomach infection	Rejected

SELECT lt.leave_type_id, lt.leave_name, lt.max
SUCCESS · Params: [] · 9 row(s) · 18.94 ms Parameterized (?) History (13)

Figure 11.2: Leave Management screen with filterable leave requests

11.3 Attendance

The Attendance screen presents the joined attendance records (38 rows) with the date, employee, department, shift with its timings, check-in and check-out times, status, and remarks. Filter tabs allow the records to be limited to Present, Late, Half-Day, Absent, or On Leave, and a date picker restricts the list to a specific day. Employees on night shifts who have not yet checked out are shown with an empty check-out field.

Attendance Records Joined View (38 rows)

SQL: SELECT a.*, e.first_name, e.last_name, d.dept_name, s.shift_name
FROM Attendance a INNER JOIN Employee e ... INNER JOIN Shift s

Search employee, dept mm/dd/yyyy

ALL Present Late Half-Day Absent On Leave

ID	Date ↑↓	Employee (FK emp_id)	Department	Shift (FK shift_id)	Check-In	Check-Out	Status ↑↓	Remarks
#18	2026-10-07	Vikram Desai (ID: 203)	Human Resources & Ops	General Day Shift (09:00-17:30)	—	—	On Leave	Annual vacation
#17	2026-10-07	Arjun Mehta (ID: 201)	Platform Engineering	General Day Shift (09:00-17:30)	08:45:00	18:10:00	Present	Overtime for sprint release
#16	2026-10-07	Zoya Khan (ID: 116)	Information Security	Night Operations Shift (22:00-06:30)	22:25:00	—	Late	Night shift in progress, arrived late
#15	2026-10-07	Siddharth Joshi (ID: ...)	Information Security	Night Operations Shift	21:58:00	—	Present	Night shift in progress

SELECT s.shift_id, s.shift_name, s.start_time, SUCCESS · Params: [] · 6 row(s) · 16.96 ms Parameterized (?) History (21) ^

Figure 11.3: Attendance records joined view with status filters

11.4 Shift

The Shift screen lists the six defined shifts with their start time, end time, grace period, and the number of active assignments, which is obtained with a LEFT JOIN so that unused shifts are still shown. A delete action is available for each shift, and the database prevents deletion of a shift that still has employees assigned to it.

Shift Records (6 rows)

Search shift name...

shift_id (PK)	Shift Name	Start Time	End Time	Grace Period	Active Assignments (LEFT JOIN)	Action
#1	General Day Shift	09:00:00	17:30:00	15 mins	19 assignment(s)	Delete
#2	Early Morning Shift	06:00:00	14:30:00	10 mins	3 assignment(s)	Delete
#3	Afternoon Swing Shift	14:00:00	22:30:00	15 mins	3 assignment(s)	Delete
#4	Night Operations Shift	22:00:00	06:30:00	20 mins	2 assignment(s)	Delete
#5	Weekend Support Shift	08:00:00	16:00:00	0 mins	2 assignment(s)	Delete
#6	Flexi Hours Shift	10:00:00	18:30:00	120 mins	2 assignment(s)	Delete

Figure 11.4: Shift records with active assignments count

12. Implementation Details

12.1 Technology Stack

Layer	Technology	Responsibility
Presentation (Frontend)	Python (Streamlit / Flask / Tkinter)	Forms, tables, dashboards, and filters shown to the user
Application (Middleware)	Node.js with Express	REST API endpoints, input validation, business rules
Data Access	mysql2 driver with connection pool	Executes parameterized SQL statements
Database	MySQL 8.x (PostgreSQL compatible)	Persistent storage, constraints, and indexes

12.2 System Architecture

The application follows a three-tier architecture. The user interacts with the Python frontend, which sends HTTP requests to the Node.js API. The API validates the request, runs a parameterized SQL statement against MySQL through a connection pool, and returns the result as JSON, which the frontend then renders as tables, metric cards, or messages. Keeping the SQL in the middleware layer means that the database credentials are never exposed to the browser and that all access goes through one controlled gateway.

```
[ Python UI ] --HTTP/JSON--> [ Node.js API ] --SQL--> [ MySQL DB ]
  Streamlit/Flask   Express routes (mysql2)   5 related tables
      ^               |               |
      +----- JSON <-----+<---- result rows -----+
```

12.3 Database Connectivity

The Node.js backend connects to MySQL using the mysql2 library with a connection pool, so that connections are reused across requests. Credentials are read from environment variables (a .env file that is excluded from version control) rather than hard-coded.

```
// db.js
require('dotenv').config();
const mysql = require('mysql2/promise');

const pool = mysql.createPool({
  host: process.env.DB_HOST || 'localhost',
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME || 'workforce_db',
  waitForConnections: true,
  connectionLimit: 10,
});

module.exports = pool;
```

12.4 Key Code Snippets

(a) API server and employee listing (Node.js)

```
// server.js
const express = require('express');
const pool = require('./db');
const app = express();
app.use(express.json());

// List employees with department and shift names (JOIN)
app.get('/api/employees', async (req, res) => {
  const [rows] = await pool.query(
    SELECT e.Emp_ID, e.Name, e.Email, d.Dept_Name, s.Shift_Name
    FROM Employee e
    JOIN Department d ON e.Dept_ID = d.Dept_ID
    JOIN Shift s ON e.Shift_ID = s.Shift_ID
    ORDER BY e.Emp_ID`);
  res.json(rows);
});

app.listen(3000, () => console.log('API running on port 3000'));
```

(b) Clock-in and clock-out with business rules (Node.js)

Clock-in refuses a second open record for the same employee on the same day, and clock-out only updates a record that is still open (Clock_Out IS NULL). Both use placeholders (?) so user input is never concatenated into SQL, which prevents SQL injection.

```
app.post('/api/attendance/clock-in', async (req, res) => {
  const { empId } = req.body;
  const [open] = await pool.execute(
    'SELECT Log_ID FROM Attendance WHERE Emp_ID = ? AND `Date` = CURDATE() AND Clock_Out IS NULL',
    [empId]);
  if (open.length) return res.status(409).json({ error: 'Already clocked in' });

  await pool.execute(
    "INSERT INTO Attendance (Emp_ID, `Date`, Clock_In, Status) VALUES (?, CURDATE(), CURTIME(), 'Present')",
    [empId]);
  res.status(201).json({ message: 'Clock-in recorded' });
});

app.put('/api/attendance/clock-out', async (req, res) => {
  const [r] = await pool.execute(
    'UPDATE Attendance SET Clock_Out = CURTIME() WHERE Emp_ID = ? AND `Date` = CURDATE() AND Clock_Out IS NULL',
    [req.body.empId]);
  if (!r.affectedRows) return res.status(404).json({ error: 'No active clock-in' });
  res.json({ message: 'Clock-out recorded' });
});
```

(c) Leave request submission and approval (Node.js)

```
app.post('/api/leaves', async (req, res) => {
  const { empId, leaveType, startDate, endDate } = req.body;
  if (endDate < startDate)
    return res.status(400).json({ error: 'End date before start date' });

  await pool.execute(
    "INSERT INTO Leave_Request (Emp_ID, Leave_Type, Start_Date, End_Date, Approval_Status) VALUES (?, ?, ?, ?, 'Pending')",
```

```

    [empId, leaveType, startDate, endDate]);
    res.status(201).json({ message: 'Leave request submitted' });
  });

app.patch('/api/leaves/:id', async (req, res) => {
  const { status } = req.body;          // 'Approved' or 'Rejected'
  await pool.execute(
    'UPDATE Leave_Request SET Approval_Status = ? WHERE Leave_ID = ?',
    [status, req.params.id]);
  res.json({ message: 'Leave status updated' });
});

```

(d) Python frontend calling the API (Streamlit)

```

# app.py
import requests
import pandas as pd
import streamlit as st

API = "http://localhost:3000/api"
st.title("Workforce Management System")

tab1, tab2 = st.tabs(["Employees", "Mark Attendance"])

with tab1:
    data = requests.get(f'{API}/employees').json()
    st.dataframe(pd.DataFrame(data))

with tab2:
    emp_id = st.number_input("Employee ID", min_value=1, step=1)
    col1, col2 = st.columns(2)
    if col1.button("Clock In"):
        r = requests.post(f'{API}/attendance/clock-in', json={"empId": emp_id})
        st.success(r.json().get("message", r.json().get("error")))
    if col2.button("Clock Out"):
        r = requests.put(f'{API}/attendance/clock-out', json={"empId": emp_id})
        st.success(r.json().get("message", r.json().get("error")))

```

12.5 Design Decisions

- **Parameterized queries:** all dynamic values are passed as parameters, which prevents SQL injection and is why the interface labels its queries as “Parameterized”.
- **Constraints in the database:** rules such as unique email, valid status values, and valid date ranges are enforced by the database so that they hold no matter which client writes the data.
- **Connection pooling:** a pool of reusable connections reduces the overhead of opening a new connection for each request.
- **Indexes:** indexes on Attendance(Emp_ID, Date) and Leave_Request(Approval_Status) speed up the most frequent dashboard queries.
- **Separation of concerns:** the frontend never talks to the database directly; all access goes through the API.

12.6 Setup Steps

- Install MySQL, Node.js, and Python 3, then create the database by running the DDL and DML scripts in MySQL Workbench.
- Clone the GitHub repository, run “npm install” in the backend folder, create a .env file with the database credentials, and start the API with “node server.js”.
- Install the Python dependencies (for example “pip install streamlit requests pandas”) and start the interface with “streamlit run app.py”.

13. Testing

Testing was carried out at the database level (constraints and queries) and at the application level (API and interface). Each test case below lists the action performed, the expected behaviour, and the observed result.

Table 13.1: Test cases and results

TC	Test Case	Input / Action	Expected Result	Result
1	Insert valid employee	INSERT with valid Dept_ID and Shift_ID	Row inserted; Emp_ID auto-generated	Pass
2	Duplicate email	INSERT employee with an existing Email	Rejected: duplicate entry (UNIQUE)	Pass
3	Invalid foreign key	INSERT employee with Dept_ID = 99	Rejected: foreign key constraint fails	Pass
4	NULL in mandatory field	INSERT employee with NULL Name	Rejected: column cannot be null	Pass
5	Clock-in	POST /attendance/clock-in for a valid employee	Attendance row created with Clock_Out NULL	Pass
6	Duplicate clock-in	Clock-in again without clocking out	HTTP 409: Already clocked in	Pass
7	Clock-out	PUT /attendance/clock-out	Clock_Out set to current time	Pass
8	Clock-out without clock-in	Clock-out for an employee with no open record	HTTP 404: No active clock-in	Pass
9	Invalid leave dates	Leave with End_Date earlier than Start_Date	Rejected by CHECK constraint / API validation	Pass
10	Leave approval	PATCH leave to Approved	Approval_Status updated; shown in Approved filter	Pass
11	Invalid leave status	UPDATE Approval_Status to "Maybe"	Rejected by CHECK constraint	Pass
12	Delete referenced shift	DELETE a shift with assigned employees	Rejected: foreign key constraint fails	Pass
13	Overtime query	Run Query 1 (Presentation-II)	Returns Fiona Gallagher, Michael Scott, Rachel Green (3 rows)	Pass
14	Shift assignment count	Open the Shift screen (LEFT JOIN query)	All 6 shifts listed, including those with zero assignments	Pass
15	Dashboard metrics	Open Dashboard for 2026-10-07	Cards show total, present, on-leave, and pending counts	Pass
16	Filter leave list	Click Pending / Approved / Rejected tabs	Only matching requests are displayed	Pass

13.2 Summary of Testing

All of the 16 test cases produced the expected results. The constraint tests (cases 2, 3, 4, 9, 11, and 12) confirm that the database itself blocks duplicate, incomplete, and orphaned data, while the application tests (cases 5 to 8 and 10) confirm that the business rules for attendance and

leave are enforced in the middleware. The query and interface tests (cases 13 to 16) confirm that the joins, aggregates, and filters return correct and complete results.

14. Conclusion and Future Enhancements

14.1 Conclusion

The Workforce Management System was successfully designed and implemented. The normalized database structure effectively handles multi-department logic without data anomalies. The integration of Node.js and Python ensures that HR administrators can manipulate and retrieve complex data through simple user interfaces.

Through this project, the main concepts of a database management system were applied end to end: the problem was modelled with an ER diagram, mapped to a relational schema, normalized to 3NF, implemented with SQL DDL and DML, and queried with joins and aggregates. The circular dependency between departments and their managers was resolved using a deferred foreign key, and integrity was protected through primary keys, foreign keys, unique, not-null, and check constraints. The result is a system that replaces error-prone manual registers with a reliable, queryable, and extensible HR backend.

14.2 Future Enhancements

- **Hardware Automation:** Integrating IoT modules (such as ESP32 microcontrollers or RFID scanners) at entry gates to directly trigger Clock_In and Clock_Out SQL queries in real-time.
- **Payroll Integration:** Linking attendance logs and overtime statuses directly to an automated salary calculation module.
- **Role-based access control:** Separate login roles for employees, managers, and HR administrators, with managers approving leaves for their own department only.
- **Notifications:** Email or mobile alerts for leave approvals, late arrivals, and missed clock-outs.
- **Reporting and analytics:** Exportable monthly reports and charts, such as attendance trends and leave balances per employee.
- **Cloud deployment:** Hosting the database and API on a cloud platform with automated backups for multi-branch organizations.

15. References

- [1] Elmasri, R., and Navathe, S. B. Fundamentals of Database Systems, 7th Edition. Pearson, 2016.
- [2] Silberschatz, A., Korth, H. F., and Sudarshan, S. Database System Concepts, 7th Edition. McGraw-Hill Education, 2019.
- [3] Codd, E. F. “A Relational Model of Data for Large Shared Data Banks.” Communications of the ACM, 13(6), 1970, pp. 377–387.
- [4] Oracle Corporation. MySQL 8.0 Reference Manual. <https://dev.mysql.com/doc/refman/8.0/en/>
- [5] The PostgreSQL Global Development Group. PostgreSQL Documentation. <https://www.postgresql.org/docs/>
- [6] OpenJS Foundation. Node.js Documentation. <https://nodejs.org/en/docs/>
- [7] Express.js Documentation. <https://expressjs.com/>
- [8] mysql2 Library for Node.js. <https://sidorares.github.io/node-mysql2/docs>
- [9] Streamlit Documentation. <https://docs.streamlit.io/>
- [10] Pallets Projects. Flask Documentation. <https://flask.palletsprojects.com/>

16. Appendix: GitHub Repository Link

The complete source code, SQL scripts (DDL and DML), and project documentation are available in the GitHub repository below.

GitHub Repository: [GITHUB](#)

The repository contains: the SQL scripts for creating and populating the database, the Node.js API source code, the Python frontend, a README with setup instructions, and this project report.